REQUIREMENTS & ASSUMPTIONS

FUNCTIONAL REQUIREMENTS

FR1 Browse sale/product pages and see live stock status (Available / Sold out)
FR2 Add to cart; "Buy Now" creates an inventory reservation (1 unit per customer per sale)
FR3 Reservation holds stock for a TTL (5 min), then auto-releases if unpaid
FR4 Checkout and pay once per reservation (idempotent)
FR5 Successful payment produces exactly one order; order moves through fulfilment states
FR6 Failed/timed-out payment releases stock; released units become buyable again
FR7 Duplicate requests (retries, double-clicks) return the original result, never a second effect
FR8 Notifications on reserve, pay, order, ship, deliver; shipment tracking

NON-FUNCTIONAL REQUIREMENTS

Oversell
  Requirement : sold + reserved <= stock, always
  Type : STRICT GUARANTEE

Duplicates
  Requirement : One payment capture and one order per reservation
  Type : STRICT GUARANTEE

Payment-order
  Requirement : Every captured payment ends in an order or a refund
  Type : STRICT (eventual, <= 5 min)

Throughput
  Requirement : 10k rps normal; reason to 500k rps flash peak (edge-absorbed)
  Type : Target

Latency
  Requirement : Reserve API p99 < 300 ms; static/browse p99 < 100 ms
  Type : Target

Availability
  Requirement : 99.95% browse; 99.9% buy path
  Type : Target

Consistency
  Requirement : Inventory: strong (single authority).
  Orders/notifications: eventual
  Type : Design

Recovery
  Requirement : RPO = 0 for inventory/payments; RTO < 5 min
  Type : Target

Security
  Requirement : TLS, JWT auth, rate limits, PCI scope minimised (tokenised card data)
  Type : STRICT

ASSUMPTIONS & CONSTRAINTS

- One hot SKU (Product X, 100 units); 10,000 users click within seconds;

~2% duplicate requests; payment success 95%.
- Card data never touches our servers (gateway-hosted tokenisation); we store only payment tokens/refs.
- Payment gateway supports idempotency keys and a status-query API.
- Limit: 1 unit per customer per sale (stops bots hoarding).
- Reservation TTL 5 min; +2 min grace while a payment is in flight.
- Critical dependencies: Redis cluster, PostgreSQL primary, Kafka, payment gateway.

CAPACITY REASONING (FLASH SALE)

- 500k rps at the edge: ~95%+ is static/browse (CDN) or rejected at edge (rate limit, waiting room, sold-out flag).
- Only ~10k "Buy" calls reach the origin in a burst; only 100 pass the inventory gate; ~100 payments follow.
- The system is a FUNNEL: 500k -> 10k -> 100 -> ~95 orders.